

Arbeitsexposé

KI-gestützte 3D-Rekonstruktion linearer Infrastruktur: Evaluierung einer Docker-basierten Scan-to-BIM Pipeline mittels SAM 3 und Gaussian Splatting

Bachelor- und Projektarbeit

19. Mai 2026

1 Einleitung und Forschungsfrage

Die präzise Erfassung schwer zu modellierender, extrem dünner Objekte (wie Stromerkabel in Baugräben) stellt für klassische photogrammetrische Verfahren (SfM) eine große Herausforderung dar. Ziel dieser Arbeit ist die Entwicklung und Evaluierung eines SScan-to-BIMWorkflows, der die Vorzüge der 2D-KI-Segmentierung (Segment Anything Model 3) mit modernsten 3D-Rekonstruktionsmethoden (3D Gaussian Splatting via SuGaR) verbindet.

Wie aktuelle Forschungen belegen, erzeugen 3D Gaussian Splatting Algorithmen durch mathematische Wahrscheinlichkeitsfunktionen hochgradig detailreiche, fotorealistische Visualisierungen bei geringer Rechenlast. Sie werden zunehmend im Ingenieurwesen und zur Generierung von Orthofotos (z.B. Integration in ArcGIS Pro) eingesetzt. Jedoch fehlt 3DGS-Modellen die inhärente Georeferenzierung. Die zentrale Forschungsfrage lautet daher: *Lässt sich die 3D-Rekonstruktion linearer Infrastruktur durch eine vorgelagerte 2D-Segmentierung und ein objektspezifisches 3DGS-Training (Segment-then-Splat) so optimieren und georeferenzieren, dass strenge ingenieurstechnische Toleranzen eingehalten werden?*

2 Praxisbezug und Erfolgskriterien

Die Pipeline wird direkt für den Anwendungsfall der TenneT (Übertragungsnetzbetreiber) konzipiert.

- **Geometrische Toleranz:** Eine Abweichung von maximal ± 10 cm in Lage und Höhe gegenüber der realen Kabeltrasse.
- **Georeferenzierung & GIS-Integration:** Das finale 3D-Modell muss präzise in einem UTM-Koordinatensystem vorliegen, um den direkten Import in Geoinformationssysteme (z.B. ArcGIS Pro) zu gewährleisten.

- **Wirtschaftliche Evaluierung:** Eine Kostengegenüberstellung des Drohnen- und Serververrechnungsaufwands im Vergleich zur klassischen, terrestrischen Vermessung.

3 Methodik und Architektur (Workflow v3)

Der Ansatz basiert auf einer klaren Trennung von 2D-Bildverarbeitung und 3D-Meshing.

3.1 Software-Repositories

- **SAM 3 (Meta):** Zuständig für Concept-Prompting und Video-Tracking.
- **COLMAP:** Generierung der Kameraposen auf Basis der *unmaskierten* Originalbilder. Die Pipeline nutzt ausschließlich das schnelle *Structure from Motion* (SfM) von COLMAP. Die rechenintensive dichte Rekonstruktion (MVS) wird vollständig übersprungen.
- **Segment-then-Splat (STS):** Übernimmt das objektspezifische 3DGS-Training (Lu et al., NeurIPS 2025). STS teilt die initialen COLMAP-Gaussians anhand der SAM 3 Masken in objektspezifische Sets auf und trainiert diese mit einem dedizierten Object-specific Loss ($\mathcal{L}_{obj}$), ohne die Ground-Truth-Bilder zu manipulieren. Das Ergebnis ist ein vollständiges 3DGS-Modell, in dem jeder Gaussian eine Object-ID trägt. SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2.
- **SuGaR:** Surface extraction with Gaussian Splatting. Übernimmt den vortrainierten STS-Checkpoint und wendet geometrische Regularisierung (DN-Consistency, Normal Alignment) sowie Poisson Surface Reconstruction an. SuGaR zwingt die von STS bereits sauber zugewiesenen Gaussians dazu, sich flach an die reale 3D-Oberfläche anzuschmiegen, und extrahiert daraus ein hochqualitatives Mesh.
- **DGtal:** Open-Source Geometrie-Bibliothek zur finalen Centerline-Extraktion. Sie wandelt die Oberfläche des Kabel-Sub-Meshes in eine räumliche 1D-Kurve (Centerline bzw. Scheitelachse mit lokalen Koordinaten) um.

3.2 Pipeline-Struktur: Temporale Konsistenz vs. SAHI

Für die Erkennung extrem dünner Kabel in hochauflösenden 4K-Bildern bietet sich theoretisch *Slicing Aided Hyper Inference* (SAHI) an. SAHI zerschneidet das Bild in kleinere Teilbereiche (Patches), wodurch verhindert wird, dass dünne Kabel beim Downscaling auf die native Modellauflösung von SAM 3 "verschluckt" werden.

Dennoch wurde der primäre Fokus für diese Pipeline bewusst auf den **SAM 3 Video Predictor** gelegt. Der methodische Nachteil von SAHI ist die fehlende temporale Konsistenz: Da SAHI jedes Einzelbild isoliert betrachtet, "flackern" die erzeugten Masken von Frame zu Frame, wenn sich Lichteinfall oder Blickwinkel leicht ändern. 3D Gaussian Splatting erfordert jedoch zwingend formstabile, konsistente Masken über alle Kameraperspektiven hinweg, da sonst fehlerhafte Artefakte in den 3D-Raum projiziert werden. Der SAM 3 Video Predictor löst dieses Problem durch ein Memory-Modul, welches die Kabel-Pixel über die Zeit hinweg verfolgt und dadurch flackerfreie, temporär konsistente

Masken liefert. SAHI dient in dieser Pipeline lediglich als Fallback-Route für spezifische Bildausschnitte, in denen das temporale Tracking abreißen sollte.

4 Docker-Infrastruktur und Orchestrierung

Ein Kernproblem moderner KI-Pipelines sind tiefgreifende Versionskonflikte zwischen den benötigten Nvidia-Grafikkartentreibern (CUDA). Dieses Problem wird durch eine `docker-compose` Architektur vollständig gelöst. Die Pipeline isoliert die Anwendungen in voneinander getrennte Container, während alle erzeugten Daten über **Docker Bind Mounts (Volumes)** zentral und persistent auf dem Host-System (Windows/Linux) gespeichert werden.

Um die Automatisierung zu maximieren, wird die Pipeline über ein zentrales Master-Skript (*One-Click-Orchestration*) gesteuert. Dieses Skript stößt die Container sequenziell an, muss jedoch für manuelle Arbeitsschritte pausieren. Es kommen fünf dedizierte Container zum Einsatz:

- **Container A (Vorverarbeitung):** Beinhaltet PyTorch 2.x und SAM 3. Benötigt das Nvidia Container Toolkit (CUDA ≥ 12.6).
- **Container B (SfM):** Ein offizielles NVIDIA-Image für COLMAP (CUDA 11.8 oder 12.1). *Nach Ausführung dieses Containers pausiert das Master-Skript automatisch (Breakpoint). Der Nutzer öffnet CloudCompare auf dem Host-System, pickt die GCPs und speichert die Transformationsmatrix. Anschließend wird das Skript mit "Enter" fortgesetzt.*
- **Container C (Object-specific 3DGS):** Beinhaltet Segment-then-Splat (STS). Empfängt die SAM 3 Masken und die COLMAP-Kameraposen und führt das objektspezifische 3DGS-Training mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$) durch. Benötigt CUDA 12.1 (PyTorch 2.3.1). SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2 als Masken-Frontend.
- **Container D (Meshing):** Beinhaltet SuGaR. Lädt den vortrainierten STS-Checkpoint und wendet die geometrische Regularisierung (DN-Consistency) sowie Poisson Surface Reconstruction an. Wegen tiefer Abhängigkeiten zum `diff-gaussian-rasterization` Modul wird hier strikt CUDA 11.8 erzwungen. Das intern geforderte Conda-Environment wird systemweit über eine Anpassung der `ENV_PATH` Variable im Dockerfile nativ zugänglich gemacht.
- **Container E (Post-Processing):** Ein reiner **CPU-Container** (ohne GPU/CUDA-Abhängigkeit). Beinhaltet die C++ Bibliothek DGtal sowie die GDAL-Tools (`ogr2ogr`). Durch die Auslagerung dieser CPU-basierten Nachbearbeitung in einen eigenen Microservice bleibt die Architektur extrem leichtgewichtig, da keine massiven Nvidia-Images geladen werden müssen.

5 Risikomanagement und Datenhandling

Drei kritische Punkte erfordern besondere Aufmerksamkeit:

1. **Speichermanagement:** Drohnenvideos in 4K bei 60 FPS erzeugen massive Datenmengen. Das Konzept sieht vor, dass diese temporär im nativen Linux-Dateisystem liegen und nach der 6 FPS Extraktion gelöscht werden, um I/O-Engpässe unter WSL2 zu vermeiden.

2. **Das Georeferenzierungs-Paradoxon:** Da die objektspezifische Selektion (STS) nur das Kabel-Mesh isoliert, verschwinden im finalen 3D-Modell auch die am Boden ausgelegten Passpunkte (GCPs). Eine Post-Training Transformation des Modells durch manuelles Anklicken ist somit nicht anwendbar. Die Lösung ist eine *Pre-Training Georeferenzierung*: Die Open-Source-Software **CloudCompare** bietet hierfür ein ideales GUI. Da CloudCompare intern mit 32-Bit Single-Precision-Gleitkommazahlen arbeitet, führt eine direkte Eingabe von großen globalen UTM-Koordinaten ($> 10^5$) zu dramatischem Präzisionsverlust (1–10 cm). Daher wird eine *relative Georeferenzierung* durchgeführt: Ein ausgewählter GCP dient als lokaler Ursprung (Ankerpunkt) mit den Koordinaten (0, 0, 0), und alle anderen GCPs werden relativ zu diesem berechnet ($GCP_{\text{relativ}} = GCP_{\text{UTM}} - GCP_{\text{Anker}}$). Man lädt die unmaskierte, spärliche SfM-Punktwolke aus COLMAP in CloudCompare, ordnet die Passpunkte den relativen GCP-Koordinaten manuell zu (*Point Picking*) und berechnet eine 4x4-Transformationsmatrix (Kombination aus 3x3-Rotation, Translation und optionaler Skalierung). SuGaR berechnet danach parallel das rohe, lokale 3D-Mesh. Anstatt jedoch das rechenintensive Millionen-Polygon-Mesh in globale Koordinaten zu überführen, wird zunächst DGtal auf dem lokalen Mesh ausgeführt. Die resultierende lokale Centerline (bestehend aus wenigen hundert Punkten) wird in Container E über ein kurzes Python-Hilfsskript mittels der 4x4-Transformationsmatrix multipliziert, um die Rotation und Skalierung anzuwenden, und danach durch Addition des Anker-GCPs in die globalen UTM-Koordinaten überführt.

3. **[NEUE ERKENNTNIS: Datenformate & GIS-Export]**

ArcGIS Pro unterstützt .ply-Geometrien nicht nativ als Feature-Layer. Zudem verursacht das .obj-Format bei großen UTM-Koordinaten signifikante Präzisionsverluste (*Vertex Jittering*), da es standardmäßig als *Single Precision* verarbeitet wird. Die Pipeline splittet den Datenexport daher gezielt in zwei Endprodukte auf:

- **Das analytische Endprodukt (Centerline):** Da DGtal (als diskrete Geometrie-Bibliothek) im lokalen Raum um den Ursprung (0,0,0) am präzisesten und speichereffizientesten arbeitet, wird die Centerline zuerst auf dem lokalen .ply-Mesh berechnet. Ein Python-Hilfsskript in Container E wendet die 4x4-Transformationsmatrix an, addiert den globalen Ankerpunkt und gibt eine georeferenzierte CSV-Datei aus. Die Konvertierung dieser CSV-Datei in standardisierte GIS-Formate (wie GeoJSON) erfolgt danach Open-Source über den **GDAL/ogr2ogr Post-Processing Container**.
- **Das visuelle Endprodukt (BIM-Mesh):** Falls TenneT das Kabel zusätzlich als 3D-Objekt visualisieren möchte, wird das lokale .obj-Mesh vorab (in CloudCompare oder über ein Python-Skript) mit dem Rotationsteil der 4x4-Matrix rotiert und skaliert. Anschließend wird die geänderte Geometrie in ArcGIS Pro über das nativ integrierte Geoprocessing-Tool *Import 3D Files* eingelesen

und der Einfügapunkt (Insertion Point / Global Shift) direkt auf die UTM-Koordinaten des Anker-GCPs gesetzt, was den Präzisionsverlust des .obj-Formats elegant umgeht.

6 Wissenschaftliche Evaluationsmetriken

Für die wissenschaftliche und ingenieurstechnische Validierung der Pipeline werden zwei Kategorien von Metriken eingesetzt: geometrische Metriken für die Lagegenauigkeit des Endprodukts und photometrische Metriken für die visuelle Rekonstruktionsqualität des 3DGS-Modells.

6.1 Geometrische Metriken (Lage- und Höhengenaugkeit)

Um nachzuweisen, dass die extrahierte Centerline die geforderte Toleranz von ± 10 cm einhält, werden die Abweichungen zu den hochpräzisen GNSS-Referenzmessungen quantifiziert.

- **B-Spline-Referenzkurve:** Die GNSS-Referenzmessungen liegen als diskrete, dreidimensionale Koordinatenpunkte $P_i = (x_i, y_i, z_i)$ vor. Ein direkter Punkt-zu-Punkt-Vergleich mit der dichter diskretisierten DGtal-Centerline ist fehleranfällig, da die Punkte nicht exakt korrespondieren. Daher wird eine kontinuierliche dreidimensionale **Basis-Spline-Kurve (B-Spline)** $S(t)$ durch die GNSS-Punkte interpoliert bzw. angenähert. Der B-Spline garantiert eine mathematisch glatte (zweifach stetig differenzierbare, C^2 -konsistente) Referenzachse des realen Kabels.
- **RMSE (Root Mean Square Error):** Der RMSE berechnet den quadratischen Mittelwert der euklidischen Abstände zwischen den Punkten C_j der extrahierten DGtal-Centerline und ihren jeweils orthogonalen Projektionen auf die B-Spline-Referenzkurve $S(t)$:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{j=1}^N \min_t \|C_j - S(t)\|^2}$$

Der RMSE gibt Aufschluss über die durchschnittliche systematische und stochastische Abweichung über den gesamten Verlauf der Trasse.

- **Hausdorff-Distanz:** Während der RMSE die mittlere Abweichung beschreibt, identifiziert die **Hausdorff-Distanz** d_H den worst-case Fehler (maximale lokale Abweichung). Sie ist definiert als das Maximum der minimalen Abstände zwischen der extrahierten Kurve C und der Referenzkurve S :

$$d_H(C, S) = \max \left\{ \sup_{c \in C} \inf_{s \in S} \|c - s\|, \sup_{s \in S} \inf_{c \in C} \|c - s\| \right\}$$

Für den Praxisbezug (TenneT) ist die Hausdorff-Distanz die entscheidende Metrik: Nur wenn $d_H(C, S) \leq 10$ cm gilt, ist garantiert, dass das Kabel an keiner einzigen Stelle der Trasse die ingenieurstechnische Toleranzgrenze überschreitet.

6.2 Photometrische Metriken (Rekonstruktionsqualität)

Diese Metriken messen ausschließlich die optische Qualität der 3D-Rekonstruktion. Sie werden **während des Trainings von Segment-then-Splat (STS)** auf gehaltenen Testbildern (Novel View Synthesis) berechnet, um sicherzustellen, dass die Geometrie der Szene visuell korrekt und ohne Artefakte erfasst wurde. Sie haben jedoch keinen direkten Bezug zur finalen UTM-Georeferenzierung.

- **PSNR (Peak Signal-to-Noise Ratio):** Misst das Verhältnis zwischen dem maximal möglichen Signalwert und dem störenden Rauschen im gerenderten Bild im Vergleich zum Ground-Truth-Foto. Der Wert wird logarithmisch in Dezibel (dB) angegeben. Höhere Werte (typischerweise > 28 dB im 3DGS) deuten auf eine präzise Rekonstruktion hin, wobei PSNR sehr empfindlich auf pixelgenaue Verschiebungen reagiert.
- **SSIM (Structural Similarity Index):** Bewertet im Gegensatz zu PSNR nicht nur den Pixelabstand, sondern die strukturelle Ähnlichkeit, Helligkeit und den Kontrast in lokalen Bildbereichen. SSIM liegt zwischen 0 und 1 (wobei 1 eine perfekte strukturelle Übereinstimmung beschreibt) und entspricht besser der menschlichen visuellen Wahrnehmung.
- **LPIPS (Learned Perceptual Image Patch Similarity):** Nutzt ein tiefes neuronales Netzwerk (z.B. VGG), um die perzeptuelle Ähnlichkeit auf Feature-Ebene zu vergleichen. Ein niedrigerer LPIPS-Wert zeigt an, dass das Rendering für das menschliche Auge (insbesondere bezüglich feiner Texturen und Schärfe) dem Originalfoto gleicht. Dies ist entscheidend, um zu prüfen, ob dünne Linienstrukturen verschwommen sind.

7 Zeit- und Scope-Planung

Projektarbeit (PA) – Fokus Machbarkeit (4 Wochen):

- Konfiguration der Docker-Architektur (Container A, B, C, D & E).
- Master-Skript Orchestrierung inkl. manuellem Breakpoint (CloudCompare).
- **Georeferenzierungs-Pipeline:** Implementierung der Transformationslogik (Berechnung der 4x4-Ähnlichkeitstransformationsmatrix in CloudCompare und automatisierte Transformation der extrahierten Centerline in Container E).
- Nachweis der Funktionalität der Segment-then-Splat Pipeline (Objekt-Loss) und Meshing (SuGaR) am Testdatensatz (Aluabspernung).
- *Optional:* Erste Voruntersuchungen zu SAM 2/3 und SAHI Tiling.

Bachelorarbeit (BA) – Fokus Skalierung & Metrik (10 Wochen):

- **Feldarbeit:** Drohnenflug sowie parallele GNSS-Referenzmessung der ausgelegten GCPs und der Kabel-Scheitelachse (*Centerline*) an der realen Trasse.
- Centerline-Extraktion via DGtal auf dem realen Kabel-Sub-Mesh.

- **Wissenschaftliche Evaluation (Metriken):** Berechnung des Root Mean Square Error (**RMSE**) und der maximalen **Hausdorff-Distanz** der extrahierten 3DGS-Centerline gegenüber einer interpolierten **Basis-Spline-Kurve (B-Spline)** aus den diskreten GNSS-Referenzmessungen, um die Einhaltung der ± 10 cm Toleranz nachzuweisen.
- Wirtschaftlichkeitsanalyse für TenneT.

Details der Parameterstudie und Sensitivitätsanalyse (BA)

Um die Robustheit und Präzision der entwickelten Pipeline unter realen Bedingungen nachzuweisen, wird im Rahmen der Bachelorarbeit eine systematische Parameterstudie durchgeführt. Dabei werden folgende Einflussfaktoren evaluiert:

- **GCP-Konfiguration (Anzahl und Verteilung):** Vergleich einer minimalen Konfiguration (3 GCPs) mit erweiterten Setups (5 bis 7 GCPs). Zudem wird die geometrische Anordnung untersucht: Lineare Platzierung (entlang des Kabelgrabens) versus stufenförmige/Zickzack-Verteilung (zur Stabilisierung der Rotationsachse um die Trassenachse).
- **Aufnahmeparameter (Flughöhe und Neigungswinkel):** Evaluierung des Einflusses der Flughöhe (5 m vs. 10 m vs. 15 m) auf die Ground Sampling Distance (GSD) und die Segmentierungsqualität. Vergleich von Nadir-Aufnahmen (90°) mit Oblique-Aufnahmen (15° – 30° Neigung), um die zylindrische Geometrie der Kabelunterkante rauscharm zu rekonstruieren.
- **Segmentierungs- und Tracking-Modelle (Frontend-Vergleich):** Systematischer Vergleich zwischen dem SAM 3 Video Predictor, SAM 2 und SAHI Tiling (Slicing). Untersuchung des Einflusses der Frame-Extraktionsrate (z. B. jeder 5. vs. 10. vs. 20. Frame) auf die Rechengeschwindigkeit und die temporale Maskenkonsistenz zur Vermeidung von 3D-Rekonstruktionsfehlern.
- **Rekonstruktions- und Meshingparameter (STS & SuGaR):** Parametertuning der Poisson-Rekonstruktionstiefe (*Poisson Depth* 7, 8, 9 und 10) in SuGaR, um den optimalen Kompromiss zwischen der Erkennung des realen Kabeldurchmessers und der Rauschunterdrückung zu finden.
- **Kurvenglättung (DGtal-Postprocessing):** Vergleich verschiedener Glättungsalgorithmen (z. B. B-Spline-Approximation oder gleitender Mittelwert) zur Rauschfilterung der extrahierten 3D-Kurve.

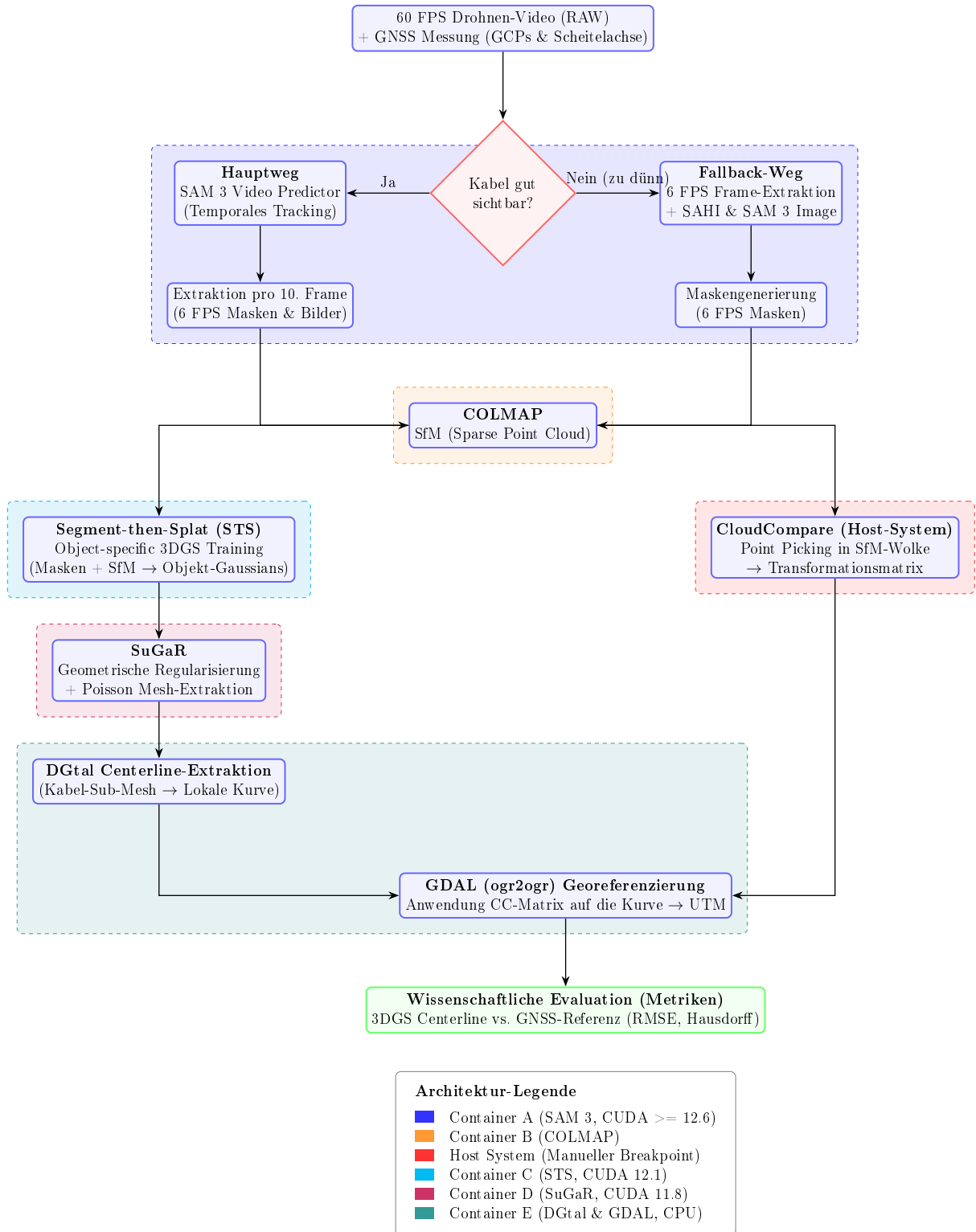


Abbildung 1: Microservice-Architektur (v4): 5-Container-Pipeline mit Segment-then-Splat für objektspezifisches Training und SuGaR für geometrisches Meshing.

8 Anhang: Single Source of Truth (SSOT) – Pipeline Installation

Dieses Dokument dient als zentrale Wahrheit (SSOT) für die Installation und Orchestrierung der gesamten 5-Container-Pipeline. Es fasst alle Hard- und Software-Abhängigkeiten zusammen, um Reproduzierbarkeit zu gewährleisten.

Hardware-Grundvoraussetzungen

- **GPU:** CUDA-kompatible Nvidia GPU (Compute Capability 7.0+).
- **VRAM:** 24 GB (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).
- **OS:** Windows 10/11 mit WSL2 oder natives Ubuntu Linux 22.04.

Container A: SAM 3 (Segmentierung)

Zuständig für die 2D-Maskierung des Drohnenvideos. Dieser Container nutzt modernste Meta-Abhängigkeiten.

Requirements (laut offizieller Vorgabe):

- Python 3.10 or higher
- PyTorch 2.4 or higher (im Setup wird PyTorch mit der passenden CUDA-Laufzeitumgebung installiert)
- CUDA-kompatible GPU mit CUDA ≥ 12.6

Installationsablauf (Conda & Pip):

```
conda create -n sam3 python=3.12
conda deactivate
conda activate sam3
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
git clone https://github.com/facebookresearch/sam3.git
cd sam3
pip install -e .
# For running example notebooks
pip install -e "[notebooks]"
# For development
pip install -e "[train,dev]"
# Optional dependencies for faster inference
pip install einops ninja && \
    pip install flash-attn --no-build-isolation
pip install git+https://github.com/ronghanghu/cc_torch.git

# Model Checkpoints (SAM 3.1) in den Container/Volume herunterladen
mkdir -p checkpoints
cd checkpoints
# Download der aktuellsten SAM 3.1 Checkpoints (Object Multiplex) via wget
wget -q https://huggingface.co/facebook/sam3-hiera-large/resolve/main/sam3.1_hiera_la
cd ..
```

Ausführung & Masken-Export (Python API):

Hier ist das Python-Skript zur automatischen temporalen Propagation des Kabel-Prompts im Video und dem anschließenden Export der binären Masken als PNG-Dateien für die 3DGS-Pipeline:

```
import os
import numpy as np
from PIL import Image
from sam3.model_builder import build_sam3_video_predictor

# 1. Video Predictor auf GPU 0 initialisieren
predictor = build_sam3_video_predictor(gpus_to_use=[0])

# 2. Session mit dem Drohnen-Video starten
video_path = "/data/video.mp4"
response = predictor.handle_request(
    request=dict(type="start_session", resource_path=video_path)
)
session_id = response["session_id"]

# 3. Prompt setzen (z. B. Text-Prompt "cable" auf Frame 0)
predictor.handle_request(
    request=dict(
        type="add_prompt",
        session_id=session_id,
        frame_index=0,
        text="cable"
    )
)

# 4. Stream-Propagation & PNG-Export in den Bind-Mount Ordner
output_dir = "/data/masks"
os.makedirs(output_dir, exist_ok=True)

# propagate_in_video liefert einen Generator für die Auswertung
for response in predictor.handle_stream_request(
    request=dict(type="propagate_in_video", session_id=session_id)
):
    frame_idx = response["frame_index"]
    outputs = response["outputs"]
    obj_ids = outputs["out_obj_ids"]
    binary_masks = outputs["out_binary_masks"] # Shape: [num_objects, H, W]

    for idx, obj_id in enumerate(obj_ids):
        mask = binary_masks[idx] # Boolesches Array [H, W]
        if mask.any():
            # Konvertierung: True -> 255, False -> 0
```

```

        mask_uint8 = (mask * 255).astype(np.uint8)
        img = Image.fromarray(mask_uint8)
        # Speichern im Format: frame_[index]_obj_[id].png
        img.save(os.path.join(output_dir, f"frame_{frame_idx:05d}_obj_{obj_id:03d}.png"))

# 5. Session sauber schließen
predictor.handle_request(
    request=dict(type="close_session", session_id=session_id)
)

```

Container B: COLMAP (Structure from Motion)

Generiert die Kameraposen und die Sparse Point Cloud aus den maskierten Drohnenbildern. Aufgrund der massiven Architekturänderungen (Einführung von GLOMAP) in Release 4.0.0, wird explizit zur stabileren Patch-Version **4.0.4** geraten, welche kritische Bugs (Speicherlecks, Race-Conditions in der Datenbank) behebt.

Quellen & Docker-Image:

- **Releases:** <https://github.com/colmap/colmap/releases>
- **Docker-Image:** `docker pull colmap/colmap:4.0.4-cuda`

Ausführung & PLY-Export (für CloudCompare):

Da COLMAP die Punktwolke standardmäßig in einem proprietären Binärformat (.bin) speichert, welches von CloudCompare nicht direkt gelesen werden kann, wird die Punktwolke direkt im Anschluss an die Rekonstruktion über den `model_converter` in eine standardisierte .ply-Datei exportiert.

```

# 1. Feature-Extraktion
colmap feature_extractor \
    --database_path /data/scene/database.db \
    --image_path /data/scene/images

# 2. Keypoint-Matching
colmap exhaustive_matcher \
    --database_path /data/scene/database.db

# 3. Structure-from-Motion (Sparse Reconstruction)
mkdir -p /data/scene/sparse
colmap mapper \
    --database_path /data/scene/database.db \
    --image_path /data/scene/images \
    --output_path /data/scene/sparse

# 4. Export der Punktwolke als PLY-Format
colmap model_converter \
    --input_path /data/scene/sparse/0 \
    --output_path /data/scene/sparse/0/points3D.ply \
    --output_type PLY

```

Breakpoint: Nach diesem Schritt pausiert das Master-Skript automatisch. Die erzeugte Datei `points3D.ply` wird auf dem Host-System in CloudCompare geladen, um die manuelle Georeferenzierung via GCP-Point-Picking durchzuführen. Nach dem Speichern der Transformationsmatrix wird das Skript fortgesetzt.

Container C: Segment-then-Splat (Object-specific 3DGS)

Führt das objektspezifische 3DGS-Training durch. STS weist jedem initialen COLMAP-Gaussian eine Object-ID zu (basierend auf den SAM 3 Masken) und trainiert mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$). Das intern vorgesehene AutoSeg-SAM2 wird durch die externen SAM 3 Masken aus Container A ersetzt.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:12.1.0-devel-ubuntu22.04`
- Python 3.10, PyTorch 2.3.1 mit CUDA 12.1
- 24 GB VRAM

Installationsablauf:

```
git clone https://github.com/luyr/Segment-then-Splat --recursive
cd Segment-then-Splat
```

```
conda create -n segment_then_splat python=3.10
conda activate segment_then_splat
pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1 \
    --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt --no-build-isolation
```

Ausführung (Preprocessing + Training):

```
# 1. Masken-Preprocessing (SAM 3 Masken aus Container A)
python ./helpers/preprocess_mask.py \
    --mask_root /data/masks \
    --out_root /data/scene/ \
    --image_path /data/scene/images

# 2. Object-specific Initialization (benötigt COLMAP-Output)
python ./helpers/object_specific_initialization.py \
    --scene_root /data/scene/

# 3. Training mit Object-specific Loss
python train.py -s /data/scene/ -m /data/output \
    --eval --iterations 40000 \
    --num_sample_objects 3 \
    --densify_until_iter 20000 \
    --partial_mask_iou 0.3
```

Wichtiger Hinweis: Das Repository ist mit dem Vermerk “(To be verified)” beim Object-Tracking markiert. Die Stabilität der Preprocessing-Skripte muss im Rahmen der Projektarbeit validiert werden.

Container D: SuGaR (Geometrische Regularisierung & Meshing)

Lädt den vortrainierten STS-Checkpoint und wendet die geometrische Regularisierung (DN-Consistency, Normal Alignment) sowie Poisson Surface Reconstruction an. Dies ist der sensitivste Container bezüglich CUDA-Versionen.

Exkurs: Conda innerhalb von Docker

Obwohl Docker bereits das Betriebssystem isoliert und Conda somit theoretisch redundant wäre, wird Miniconda in diesem Container explizit genutzt. Der Grund: 3DGS-Repositories (wie SuGaR) setzen stark auf vorkompilierte Conda-Binaries (z.B. `pytorch-cuda`) und exakte Paket-Versionen. Durch die strikte Nutzung von Conda im Docker-Container werden fatale C++ Kompilierungsfehler beim Rasterizer-Modul präventiv vermieden.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:11.8.0-devel-ubuntu22.04` (Dieses Image bringt das kompatible CUDA 11.8 SDK direkt mit).
- **C++ Compiler:** Installation via `apt-get install build-essential` (installiert GCC/G++, was unter Linux Visual Studio ersetzt und garantiert mit CUDA 11.8 kompatibel ist).
- 24 GB VRAM (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).

Installationsablauf (Dockerfile-Struktur):

```
# 1. Base Image mit CUDA 11.8 & GCC
FROM nvidia/cuda:11.8.0-devel-ubuntu22.04

# 2. System-Abhängigkeiten installieren
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y \
    build-essential \
    git \
    wget \
    curl \
    libgl1-mesa-glx \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# 3. Miniconda installieren
RUN wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O miniconda.sh
    bash miniconda.sh -b -p /opt/conda && \
    rm miniconda.sh
ENV PATH=/opt/conda/bin:$PATH

# 4. Conda Environment anlegen und Shell konfigurieren
RUN conda create --name sugar -y python=3.9
SHELL ["conda", "run", "-n", "sugar", "/bin/bash", "-c"]

# 5. PyTorch & CUDA 11.8 Bibliotheken installieren
RUN conda install pytorch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 \
```

```

pytorch-cuda=11.8 -c pytorch -c nvidia -y && \
conda install -c fvcore -c iopath -c conda-forge fvcore iopath -y && \
conda install pytorch3d==0.7.4 -c pytorch3d -y && \
conda install -c plotly plotly -y && \
conda install -c conda-forge rich plyfile==0.8.1 jupyterlab nodejs ipywidgets -y

# 6. Repositories klonen und Submodule kompilieren
WORKDIR /workspace
RUN git clone https://github.com/Anttwo/SuGaR.git && \
    cd SuGaR && \
    pip install open3d PyMCubes && \
    pip install -e . && \
    cd gaussian_splatting/submodules/diff-gaussian-rasterization/ && \
    pip install -e . && \
    cd ../simple-knn/ && \
    pip install -e .

# Conda-Umgebung im PATH systemweit als Standard setzen
ENV PATH=/opt/conda/envs/sugar/bin:$PATH

```

Wichtige Best-Practices & Parameter-Tuning (SuGaR):

Für qualitativ hochwertige Rekonstruktionen (insb. dünne Kabelstrukturen) sind folgende Parameter entscheidend (siehe SuGaR Tips):

- **Regularisierung:** Nutze zwingend `-r "dn_consistency"` für die beste Mesh-Qualität.
- **Löcher im Mesh:** Wenn die Mesh-Bereinigung zu aggressiv ist, setze `vertices_density_quantile = 0.` in `sugar_extractors/coarse_mesh.py`.
- **Ellipsoide Artefakte (Bumps):** Reduziere die Poisson-Tiefe `poisson_depth` (z.B. von 10 auf 7 oder 8 in `coarse_mesh.py`), falls einzelne Gaussians sichtbar werden.
- **Große Szenen:** Bei weiten Distanzen sollten Custom Bounding Boxes (`-bboxmin` und `-bboxmax`) genutzt werden, um die Extraktion zu fokussieren.

Container E: DGtal & GDAL (Post-Processing)

Führt die Extraktion der Centerline auf dem isolierten Kabel-Sub-Mesh durch und überführt diese in georeferenzierte GIS-Formate (GeoJSON). Da **DGtal** eine komplexe C++ Bibliothek mit zahlreichen Abhängigkeiten (Eigen, CGAL, ITK) ist, wird direkt auf dem offiziellen `dgatal:latest` Docker-Image aufgebaut.

Requirements & Repositories:

- **DGtal Release (v2.1):** <https://github.com/DGtal-team/DGtal/releases/tag/v2.1>
- **GDAL:** <https://github.com/OSGeo/gdal>
- **OGR2OGR Wrapper:** <https://github.com/wavded/ogr2ogr>

Docker Build & Ausführung:

```
# Offizielles DGtal Image bauen (im DGtal Repo Ordner)
```

```

docker build -t dgtal:latest .

# Nachinstallation von GDAL (ogr2ogr) und Python-Paketen über ein abgeleitetes Docker
FROM dgtal:latest
USER root
RUN apt-get update && apt-get install -y gdal-bin libgdal-dev python3-pip python3-num
rm -rf /var/lib/apt/lists/*
USER digital

# Interaktive Ausführung mit Bind-Mount
docker run -it -v /host/path/data:/data --user=digital dgtal-gdal:latest bash
cd /home/digital/git/DGtal/build/examples
# 1. DGtal Centerline-Extraktion auf dem lokalen Kabel-Mesh ausführen
# (Erzeugt /data/centerline_local.csv)

# 2. Ausführen des Python-Skripts zur Georeferenzierung
# (Wendet 4x4-Transformationsmatrix an und addiert UTM-Koordinaten des Anker-GCPs)
python /data/scripts/transform_centerline.py \
    --input_csv /data/centerline_local.csv \
    --matrix /data/matrix.txt \
    --anchor_gcp 32624102.5 5410204.8 123.4 \
    --output_csv /data/centerline_utm.csv

# 3. Konvertierung des UTM-Ergebnisses in standardisiertes GeoJSON (EPSG:25832)
ogr2ogr -f "GeoJSON" /data/centerline.geojson /data/centerline_utm.csv -a_srs EPSG:25

```